

República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial José Antonio Anzoátegui

Extensión - Guanta

Análisis Comparativo de Paradigmas de Programación Orientada a Objetos: Implementaciones Avanzadas en Python y PHP

Profesora:

María Parra

Bachilleres:

Franco Alvarado

Guanta 20/07/2026

DESARROLLO

1. Especificadores de Acceso y Control de Visibilidad

Los especificadores de acceso representan el pilar fundamental del encapsulamiento en la Programación Orientada a Objetos (POO). Su propósito primordial es restringir el acceso directo a los componentes internos de un objeto, protegiendo la integridad de sus datos y exponiendo únicamente una interfaz controlada.

Enfoque de Implementación: PHP vs. Python

La filosofía de diseño de ambos lenguajes aborda el control de visibilidad desde perspectivas diametralmente opuestas: **restricción estricta por compilación/interpretación (PHP)** frente a **convención y seguridad por consenso (Python)**.

Especificador	Comportamiento en PHP	Comportamiento en Python
Público (public)	Accesible desde cualquier lugar del script.	Comportamiento por defecto de cualquier atributo o método.
Protegido (protected / _)	Accesible solo dentro de la clase y por clases heredadas.	Convención visual (<code>_variable</code>). Indica un elemento de uso interno, pero es accesible externamente.
Privado (private / <code>__</code>)	Accesible única y exclusivamente dentro de la clase que lo define.	Mecanismo de <i>Name Mangling</i> (<code>__variable</code>). El intérprete transforma el nombre para dificultar su acceso externo.

Estudio de Casos en Código

Caso PHP: Control Estricto

En PHP, el motor de ejecución genera un error fatal (*Fatal Error*) si se intenta violar las reglas de visibilidad.

```
<?php
class Usuario {
    public string $nombre;
    protected string $email;
    private string $password;
    public function __construct(string $nombre, string $email, string
$password) {
        $this->nombre = $nombre;
        $this->email = $email;
        $this->password = $password;
    }
}
$user = new Usuario("Carlos", "carlos@mail.com", "secret123");
echo $user->nombre; // Permitido
// echo $user->email; // Error Fatal: Cannot access protected property
// echo $user->password; // Error Fatal: Cannot access private property
?>
```

Caso Python: Convención y *Name Mangling*

Python no posee palabras clave como `private`. Utiliza prefijos de guiones bajos. El uso de doble guion bajo activa el *Name Mangling*, modificando internamente el identificador a `__NombreClase__` atributo.

```
class Usuario:
    def __init__(self, nombre, email, password):
        self.nombre = nombre # Public
        self._email = email # Protected (Por convención)
        self.__password = password # Private (Aplica Name Mangling)

user = Usuario("Carlos", "carlos@mail.com", "secret123")
print(user.nombre) # Permitido
print(user._email) # Permitido (Python lo advierte como mala práctica,
pero no lo bloquea)

# print(user.__password) # AttributeError: 'Usuario' object has no
attribute '__password'
# Acceso forzado mediante Name Mangling (Demuestra que no es estrictamente
privado):
print(user._Usuario__password) # Imprime: secret123
```

2. Constructores y Métodos de Inicialización

El constructor es un método especial cuya función biológica dentro del ciclo de vida de un objeto es inicializar su estado basal al momento de la instanciación. Se encarga de

asignar la memoria inicial y configurar las propiedades requeridas para que el objeto sea operacional desde el primer instante.

Diferencias Arquitectónicas

PHP (public function `__construct`): Es un método mágico explícito. PHP permite la sintaxis moderna de *Constructor Promotion* (a partir de PHP 8), lo que reduce drásticamente el código repetitivo al declarar y asignar propiedades directamente en los argumentos del constructor.

Python (def `__init__`(self)): No es técnicamente el creador del objeto (ese rol le pertenece al método místico `__new__`), sino el inicializador del estado. Requiere la declaración explícita de `self` como primer parámetro para referenciar la instancia en desarrollo, una característica ausente en los métodos de PHP, donde `$this` está implícitamente disponible.

Estudio de Casos en Código

Caso PHP: Inicialización con Promoción de Propiedades

```
<?php
class ConexiónBD {
    // Constructor moderno (Maneja declaración y asignación simultánea)
    public function __construct(
        private string $host,
        private string $usuario,
        protected string $baseDatos
    ) {}
    public function obtenerInfo(): string {
        return "Conectado a {$this->baseDatos} en {$this->host}";
    }
}
$conexion = new ConexiónBD("localhost", "root", "produccion_db");
echo $conexion->obtenerInfo();
?>Caso Python: Inicialización Estándar
class ConexionBD:
    def __init__(self, host: str, usuario: str, base_datos: str):
        # Es imperativo mapear los argumentos recibidos hacia los atributos de
la instancia
        self.__host = host
        self.__usuario = usuario
        self._base_datos = base_datos
    def obtener_info(self) -> str:
        return f"Conectado a {self._base_datos} en {self.__host}"
conexion = ConexionBD("localhost", "root", "produccion_db")
```

3. Herencia y Reutilización de Código entre Clases

La herencia es un mecanismo primordial de la programación orientada a objetos que permite estructurar jerarquías de clases, facilitando la reutilización de código y la creación de relaciones de especialización ("es un"). A través de ella, una clase hija (subclase) hereda los atributos y métodos de una clase padre (superclase), reduciendo la redundancia de código y mejorando la mantenibilidad del software.

Divergencias Arquitectónicas: Herencia Simple frente a Herencia Múltiple

La diferencia fundamental entre ambos entornos radica en la capacidad de heredar de múltiples entidades:

PHP (`extends`): Implementa un modelo de **herencia simple**. Una clase solo puede heredar directamente de una única superclase. Para solventar esta limitación teórica y permitir la reutilización horizontal de código sin caer en los problemas de la herencia múltiple, PHP introduce los *Traits* a partir de la versión 5.4.

Python (`class Subclase(Superclase):`): Soporta nativamente **herencia múltiple**. Una subclase puede especificar múltiples clases base en su declaración. Para resolver los conflictos de colisión de nombres (como el clásico problema del diamante), Python utiliza el algoritmo de **Orden de Resolución de Métodos (MRO)** basado en la linealización C3, evaluando el orden de izquierda a derecha.

Estudio de Casos en Código

Caso PHP: Jerarquía Simple y Acoplamiento Explícito

En PHP, el constructor de la clase hija no invoca automáticamente al de la clase padre; es necesario llamarlo explícitamente mediante la sintaxis `parent::__construct()`.

```
<?php
class Persona {
    public function __construct(
        protected string $nombre,
        protected int $edad
    ) {}
    public function presentarse(): string {
```

```

        return "Hola, mi nombre es {$this->nombre} y tengo {$this->edad}
años.";
    }
}
// Implementación de la herencia mediante 'extends'
class Alumno extends Persona {
    private string $matricula;
    public function __construct(string $nombre, int $edad, string
$matricula) {
        // Invocación explícita al constructor padre
        parent::__construct($nombre, $edad);
        $this->matricula = $matricula;
    }
    public function obtenerHistorial(): string {
        return $this->presentarse() . " Mi matrícula estudiantil es:
{$this->matricula}.";
    }
}
$alumno = new Alumno("Sofía", 21, "A019283");
echo $alumno->obtenerHistorial();
?>

```

Caso Python: Herencia Directa y Gestión del MRO

En Python, el acceso a la superclase se gestiona de forma dinámica mediante la función proxy `super()`, la cual sigue estrictamente el orden determinado por el MRO de la instancia.

```

class Persona:
    def __init__(self, nombre: str, edad: int):
        self._nombre = nombre
        self._edad = edad

    def presentarse(self) -> str:
        return f"Hola, mi nombre es {self._nombre} y tengo {self._edad}
años."

# Implementación de la herencia pasando la clase padre como argumento
class Alumno(Persona):
    def __init__(self, nombre: str, edad: int, matricula: str):
        # Invocación de la superclase mediante super()
        super().__init__(nombre, edad)
        self._matricula = matricula

    def obtener_historial(self) -> str:
        return f"{self.presentarse()} Mi matrícula estudiantil es:
{self._matricula}."

alumno = Alumno("Sofía", 21, "A019283")
print(alumno.obtener_historial())

# Inspección del MRO de la clase Alumno
# print(Alumno.__mro__)

```

```
# Salida: (<class '__main__.Alumno'>, <class '__main__.Persona'>, <class
```

4. Clases Abstractas y Plantillas Base no Instanciables

Una clase abstracta actúa como un contrato arquitectónico o plantilla conceptual para otras clases. Su propósito no es ser instanciada directamente (lo cual genera un error en tiempo de ejecución o compilación), sino definir una interfaz común y comportamiento base parcial que las subclases concretas deben obligatoriamente implementar y extender.

Filosofía de Diseño: Nativo vs. Módulo del Sistema

PHP (abstract class): El concepto de abstracción está integrado en el núcleo del lenguaje (*Core*). Las palabras clave `abstract class` y `abstract public function` son validadas de forma estricta por el motor de PHP durante la fase de análisis del script. Si una subclase no define un método abstracto heredado, la ejecución se interrumpe inmediatamente.

Python (abc.ABC): Fiel a su filosofía de tipado dinámico (*Duck Typing*), Python no incluye clases abstractas en su sintaxis nativa elemental. En su lugar, proporciona el módulo estándar `abc` (**Abstract Base Classes**). Una clase se vuelve abstracta al heredar de `ABC`, y sus métodos se marcan mediante el decorador `@abstractmethod`. La validación ocurre en el momento del intento de instanciación.

Estudio de Casos en Código

Caso PHP: Abstracción Nativa

```
<?php
// Declaración explícita de la naturaleza abstracta de la clase
abstract class Empleado {
    public function __construct(protected string $nombre) {}
    // Contrato: toda clase hija debe definir las reglas de este método
    abstract public function calcularSalario(): float;
}
class EmpleadoTiempoCompleto extends Empleado {
    private float $salarioMensual = 3500.00;
    public function calcularSalario(): float {
        return $this->salarioMensual;
    }
}
// $anonimo = new Empleado("Error"); // Error Fatal: Cannot instantiate
abstract class Empleado
$ingeniero = new EmpleadoTiempoCompleto("Andrés");
echo "Salario de Andrés: $" . $ingeniero->calcularSalario();
?>Salario  ?>
```

```

Caso Python: Importación del Módulo abc
from abc import ABC, abstractmethod
# La clase debe heredar de la clase base abstracta ABC
class Empleado(ABC):
    def __init__(self, nombre: str):
        self._nombre = nombre
    # Decorador imperativo para forzar la implementación
    @abstractmethod
    def calcular_salario(self) -> float:
        pass
class EmpleadoTiempoCompleto(Empleado):
    def __init__(self, nombre: str):
        super().__init__(nombre)
        self.__salario_mensual = 3500.00
    def calcular_salario(self) -> float:
        return self.__salario_mensual
# TypeError: Can't instantiate abstract class Empleado with abstract method
calcular_salario
# anonimo = Empleado("Error")
ingeniero = EmpleadoTiempoCompleto("Andrés")
print(f"Salario de Andrés: ${ingeniero.calcular_salario()}")

```

5. Polimorfismo y Sobreescritura en Clases Hijas

El polimorfismo—etimológicamente "muchas formas"—es la propiedad por la cual diferentes clases de objetos pueden responder al mismo mensaje o invocación de método de maneras formalmente distintas. En los lenguajes de programación analizados, este concepto se manifiesta principalmente a través de la **sobreescritura de métodos** (*Method Overriding*), donde una subclase redefine un método heredado de su superclase para adaptar su comportamiento específico.

Filosofía del Polimorfismo: Tipado Estricto vs. *Duck Typing*

PHP (Polimorfismo de Subtipado e Interfaces): Se fundamenta en la compatibilidad de tipos estructurales. Para garantizar la seguridad del polimorfismo, PHP exige que los métodos sobreescritos mantengan la **covarianza** en los tipos de retorno y la **contravarianza** en los argumentos (Principio de Sustitución de Liskov). El polimorfismo se ejecuta asegurando que el objeto instanciado pertenezca a la jerarquía de la clase base o interfaz declarada en el tipado de los parámetros.

Python (*Duck Typing* / Tipado Pato): Se rige bajo la premisa pragmática: "*Si camina como un pato y grazna como un pato, entonces es un pato*". Python no requiere que

los objetos compartan una raíz genealógica (una clase padre común) para comportarse de forma polimórfica. Basta con que compartan la interfaz semántica (el nombre del método). La sobreescritura es implícita; redefinir un método con el mismo nombre en la clase hija reemplaza por completo la referencia en el espacio de nombres local de la instancia.

Estudio de Casos en Código

Caso PHP: Polimorfismo por Inyección de Dependencia Tipada

```
<?php
interface Notificable {
    public function enviar(string $mensaje): string;
}
class NotificacionEmail implements Notificable {
    // Sobreescritura del contrato de la interfaz
    public function enviar(string $mensaje): string {
        return "Enviando Email con el texto: '{$mensaje}'";
    }
}
class NotificacionSMS implements Notificable {
    public function enviar(string $mensaje): string {
        return "Enviando SMS con el texto: '{$mensaje}'";
    }
}
// El polimorfismo se garantiza exigiendo el tipo de la interfaz
'Notificable'
function procesarAlerta(Notificable $notificador, string $msg) {
    echo $notificador->enviar($msg) . "\n";
}
procesarAlerta(new NotificacionEmail(), "Cierre de ciclo");
procesarAlerta(new NotificacionSMS(), "Nueva nota registrada");
?>
```

Caso Python: Polimorfismo Dinámico mediante *Duck Typing*

```
class NotificacionEmail:
    # No heredan de una base común, pero implementan el mismo método
    def enviar(self, mensaje: str) -> str:
        return f"Enviando Email con el texto: '{mensaje}'"
class NotificacionSMS:
    def enviar(self, mensaje: str) -> str:
        return f"Enviando SMS con el texto: '{mensaje}'"
# Python no evalúa el tipo del objeto, sino la existencia del método
'.enviar()'
def procesar_alerta(notificador, msg: str):
    print(notificador.enviar(msg))
# Ejecución polimórfica fluida
procesar_alerta(NotificacionEmail(), "Cierre de ciclo")
procesar_alerta(NotificacionSMS(), "Nueva nota registrada")
```

CONCLUSION

El análisis comparativo detallado entre PHP y Python a lo largo de los cinco ejes rectores de la Programación Orientada a Objetos permite extraer las siguientes deducciones:

PHP adopta un enfoque de ingeniería clásico y corporativo. Su infraestructura valida de forma punitiva el control de visibilidad (`public`, `protected`, `private`), la consistencia de los constructores y la herencia simple mediante análisis estático. Es un ecosistema diseñado para minimizar los errores humanos en arquitecturas monolíticas o empresariales de gran escala mediante contratos explícitos.

Python traslada la responsabilidad del control arquitectónico directamente al desarrollador. Al carecer de barreras físicas de compilación para la privacidad (sustituyéndolas por convenciones como el guion bajo o mutaciones como el *Name Mangling*), el lenguaje prioriza la legibilidad, la velocidad de prototipado y la flexibilidad meta-programática. Su soporte para la herencia múltiple gestionada por MRO y el polimorfismo basado en *Duck Typing* descentralizan el acoplamiento jerárquico.

Mientras que PHP ha evolucionado hacia la optimización de código mediante azúcar sintáctico avanzado (como la promoción de propiedades en constructores), Python se mantiene fiel a su pureza explícita requiriendo el paso del parámetro `self`. En términos de abstracción, PHP opera de forma nativa e inmediata en su núcleo, mientras que Python delega de forma elegante este control al módulo opcional `abc`.

la elección entre ambos entornos no debe fundamentarse en capacidades técnicas abstractas, sino en los requerimientos del proyecto: PHP sobresale en sistemas donde el control estricto de tipos y las interfaces explícitas mitigan riesgos, mientras que Python es idóneo para entornos de desarrollo ágil, análisis de datos y sistemas complejos que se beneficien de la herencia múltiple y el tipado dinámico.

REFERENCIAS

PHP Architecture Reference:

The PHP Group. (2026). *Object Oriented Programming in PHP: Classes and Objects Manual*. Recuperado de <https://www.php.net/manual/en/language.oop5.php>

Python Core Software Engineering:

Python Software Foundation. (2026). *The Python Language Reference: Data Model (Section 3.3 - Special method names)*. Recuperado de <https://docs.python.org/3/reference/datamodel.html>

Teoría de Paradigmas POO:

Meyer, B. (2009). *Object-Oriented Software Construction* (2nd ed.). Prentice Hall.